



CEF488: Systems and Network Programming

LAB3: Advanced Socket Programming & Protocol Design

GROUP 8 MEMBERS

ECHUNDIP CARL-LAWRENCE	FE23A040
NSEMEN PHILLIS NANA	FE25A679
BERINYUY GILLES KILALON	FE23A024
TAMBE RAMSON	FE23A156
OBIOMA OBI FODJO	FE23A133

Lecturer: Engr Forcha Glen

2025-2026 academic year/ Semester 2

May,2026

Table of Contents

1. Honesty report	3
2. Abstract:	3
3. Objectives:.....	3
4. Pre-lab questions:.....	4
5. Methodology	4
❖ Protocol Design	4
❖ Concurrency Model.....	5
❖ Transport Protocol Choice.....	5
❖ Address Family and Dual-Stack Support	5
❖ In-Memory Storage	5
6. Implementation:.....	5
❖ TASK 1 and 2:.....	5
❖ Testing phase 1 and 2:.....	7
❖ Task3:	8
❖ Testing face for Task3	9
❖ Task 4 Implementation:.....	10
7. Conclusion	14
8. References	15

1. Honesty report

Task	Responsible member	Contributions	Reviewers
Prelab Questions	TAMBE RAMSON	NSEMEN PHILLIS NANA	OBIOMA OBI FODJO
Implementations	ECHUNDIP CARL- LAWRENCE	NSEMEN PHILLIS NANA	BERINYUY GILLES KILALON
Testing and debugging	OBIOMA OBI FODJO NSEMEN PHILLIS NANA	BERINYUY GILLES KILALON	TAMBE RAMSON
Report Writing	BERINYUY GILLES KILALON ECHUNDIP CARL- LAWRENCE	NSEMEN PHILLIS NANA	TAMBE RAMSON OBIOMA OBI FODJO

2. Abstract:

At its core, the main aim to this lab was for us to build a miniature version of a networked dictionary that lives on a server and can be queried by clients over a network, something like Redis or Memcached. We store a key retrieve it later and then go on to deleting it. Our entire lab wraps on that simple idea in a rich set of systems programming challenges given as questions:

how do two programs on a network actually talk to each other?

What language do they speak?

What happens when the network drops a packet?

How do you serve hundreds of clients at once without your server freezing?

As we come to realise this project, all these questions shall be answered.

All this and more gotten and answered using the 3 main functions: SET, GET and DEL.

3. Objectives:

Our main objective in this lab was to build a networked key-value store (like a mini Redis) that clients can query over a network to SET, GET, and DELETE data. You design a compact binary protocol defining exactly how bytes are arranged in every message sent between client and server. The server uses epoll and non-blocking sockets to handle many clients simultaneously without freezing, while supporting both TCP (reliable) and UDP (fast). `getaddrinfo()` makes our code work transparently on both IPv4 and IPv6, and socket options like `TCP_NODELAY` lets us tune performance and measure the real impact of those choices. The whole point of the lab is to demystify how real networked systems work at the byte level the same foundations underlying Redis, nginx, and virtually every high-performance server we may ever use.

4. Pre-lab questions:

1. Why is `getaddrinfo()` preferred over `gethostbyname()`? How does it support IPv6?
 - `getaddrinfo()` is thread-safe, protocol-agnostic, and returns both IPv4 and IPv6 addresses. `gethostbyname()` only handles IPv4 and uses global state.
2. Explain the purpose of the IPV6 servers?
 - IPV6 lets an IPv6 socket accept IPv4 connections via mapped addresses (e.g. `::ffff:127.0.0.1`). Setting it to 1 makes the socket IPv6-only, so you'd need two sockets for dual-stack.
3. Describe the advantages of a binary protocol over a text-based protocol (e.g., HTTP) for a key-value store.
 - Binary protocols are faster to parse (fixed-width fields, no tokenizing), more compact, and unambiguous. Text protocols like HTTP require string parsing and are larger on the wire.
4. What is the difference between SO you use each? `REUSEADDR` and `SO_REUSEPORT`? When should:
 - `SO_REUSEADDR` lets you rebind a port in `TIME_WAIT`. `SO_REUSEPORT` lets multiple sockets bind the same port (for load balancing). Use `REUSEADDR` always on servers; use `REUSEPORT` only if you want multiple processes sharing a port.
5. Why is it necessary to use non-blocking sockets when handling many concurrent clients?
 - Nagle's algorithm batches small writes to reduce packet count, but adds latency. Disable with `TCP_NODELAY` for interactive/low-latency workloads.
6. Why is it necessary to use non-blocking sockets when handling many concurrent clients? What is the role of `epoll`?

Blocking sockets stall the whole process. Non-blocking + `epoll` lets one thread watch thousands of connections and only act when data is ready.
7. In UDP, what are the potential issues with sending a request and waiting for a response? How does retransmission with exponential backoff help:
 - UDP packets can be lost or reordered with no notification. Exponential backoff
 - retransmission avoids flooding the network on retry while still recovering from loss.
8. How would you ensure that a TCP client can recover from a server restart or network failure? Wrap `connect()` in a retry loop. On `read()` returning 0 or an error, close and reconnect.

5. Methodology

❖ Protocol Design

A binary protocol was chosen over a text-based alternative for its parsing efficiency and compactness and as seen every message consists of a fixed 8-byte header carrying the total packet length and operation code, followed by null-terminated key and value strings in the payload. All multi-byte integer fields are transmitted in network byte order using `htonl()` and `ntohl()`, ensuring correctness across machines with different CPU architectures. This design gives the receiver a deterministic read strategy: read 8 bytes, extract the total length, read the remainder no tokenizing, no ambiguity.

❖ Concurrency Model

The TCP server uses a single-threaded event-driven concurrency model built on Linux's `epoll` interface with edge-triggered notification (`EPOLLET`). This was chosen over a thread-per-client model because it scales to large numbers of simultaneous connections without the memory and context-switching overhead of threads. All accepted client sockets are set to non-blocking mode using `fcntl()` with `O_NONBLOCK`, and the server loops on `epoll_wait()`, processing only connections that have data ready. Because edge-triggered mode notifies only once per new data arrival, each readable socket is drained completely until `EAGAIN` is returned before moving on.

❖ Transport Protocol Choice

Both TCP and UDP are supported to address different operational needs. TCP is used as the primary transport because it provides ordered, reliable delivery with no extra effort from the application layer, making it suitable for general-purpose key-value operations. UDP is offered as a low-latency alternative for environments where speed is prioritised over guaranteed delivery. Since UDP provides no reliability, the client implements exponential backoff retransmission: starting at a 200ms timeout, doubling on each failed attempt, up to a maximum of three retries. The server requires no state for this it simply responds to each datagram as it arrives.

❖ Address Family and Dual-Stack Support

`getaddrinfo()` was used exclusively for all address resolution on both the client and server sides, with **AI_PASSIVE** on the server and **AF_UNSPEC** on the client. This makes the code protocol-agnostic it works on IPv4, IPv6, or both without modification. The server socket is configured with `IPV6_V6ONLY` set to 0, allowing a single IPv6 socket to accept both IPv4 and IPv6 connections simultaneously, with IPv4 clients appearing as IPv6-mapped addresses.

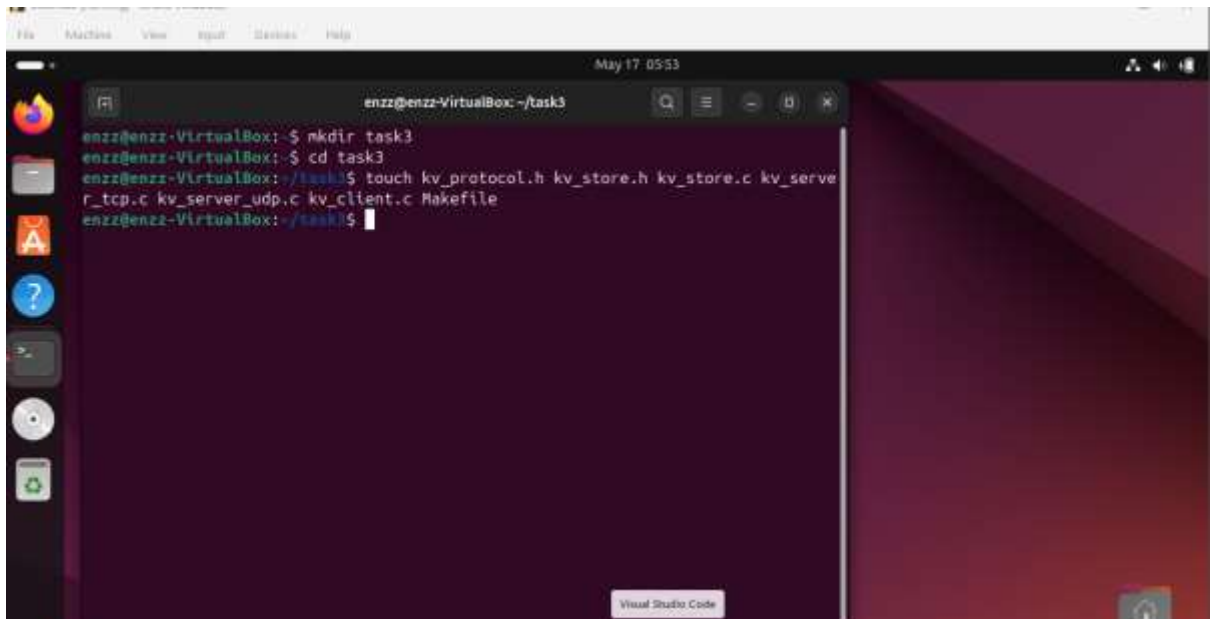
❖ In-Memory Storage

The key-value store is implemented as a hash table with 1024 buckets and linked-list chaining for collision resolution. The djb2 hash function is used for its simplicity and good distribution. Keys and values are heap-allocated via `strdup()` and freed explicitly on deletion or overwrite, keeping the implementation self-contained with no external database dependency.

6. Implementation:

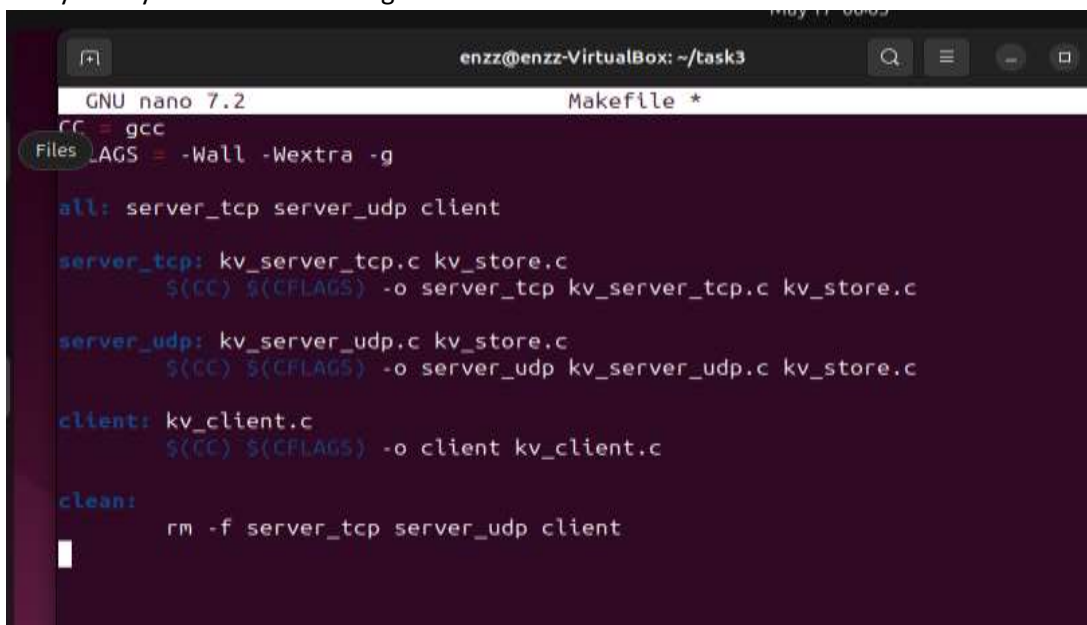
❖ TASK 1 and 2:

1. The first step of our implementation was creating several directories where we put in several codes for declaration as well as key characteristics and functions: the SET, GET and DEL operations of our mini system. The next face is creating the different files used for the implementation of this system.



```
enzz@enzz-VirtualBox: ~/task3
enzz@enzz-VirtualBox:~$ mkdir task3
enzz@enzz-VirtualBox:~$ cd task3
enzz@enzz-VirtualBox:~/task3$ touch kv_protocol.h kv_store.h kv_store.c kv_server_tcp.c kv_server_udp.c kv_client.c Makefile
enzz@enzz-VirtualBox:~/task3$
```

2. From here we code in the different files as seen in the following:
The Makefile is an automated build script it tells the compiler exactly how to turn our source code files into runnable programs, so we don't have to type long gcc commands manually every time you we make a change.



```
GNU nano 7.2 Makefile *
CC = gcc
CFLAGS = -Wall -Wextra -g

all: server_tcp server_udp client

server_tcp: kv_server_tcp.c kv_store.c
$(CC) $(CFLAGS) -o server_tcp kv_server_tcp.c kv_store.c

server_udp: kv_server_udp.c kv_store.c
$(CC) $(CFLAGS) -o server_udp kv_server_udp.c kv_store.c

client: kv_client.c
$(CC) $(CFLAGS) -o client kv_client.c

clean:
rm -f server_tcp server_udp client
```

kv_protocol.h: which is a shared file between client and server.

kv_store(): searches for an existing key first; if found, updates value. If not, prepends to the bucket list.

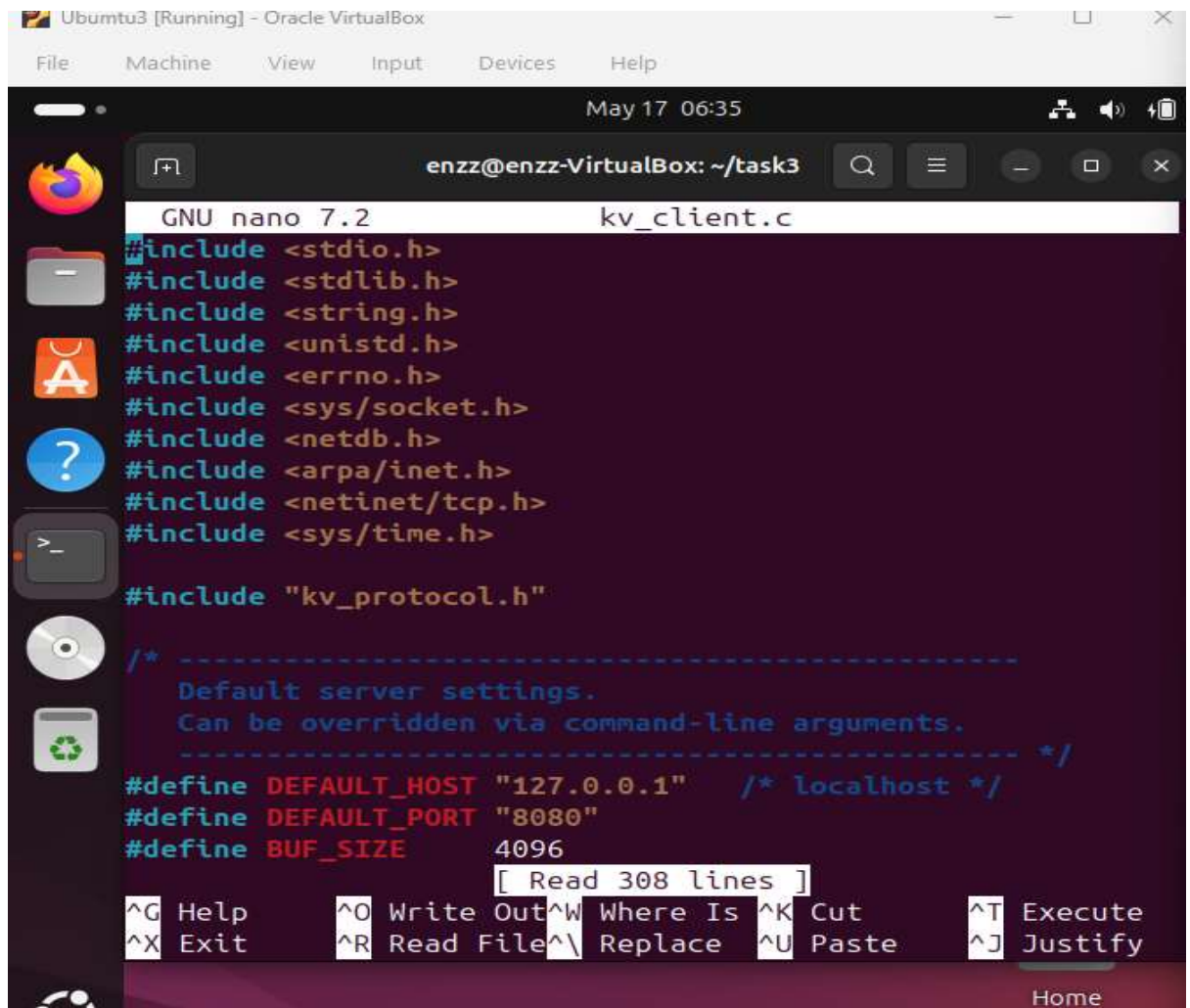
kv_server_tcp.c: This is the TCP server, monitors clients while running in the background. receives packet, parses it.

kv_server_udp.c: Still the UDP server and carries main functions as that of TCP.

kv_store.c: Stores or retrieves the data

kv_client.c: Prints result to user

In short, kv_store.c is the brain, kv_server_tcp.c is the gateway and kv_client.c is the interface.

A screenshot of a virtual machine window titled 'Ubuntu3 [Running] - Oracle VirtualBox'. The window shows a terminal with the nano 7.2 text editor open, editing a file named 'kv_client.c'. The editor's content includes several include directives for standard C libraries and network-related headers, followed by a comment block for default server settings and three preprocessor definitions for DEFAULT_HOST, DEFAULT_PORT, and BUF_SIZE. The bottom of the screen displays a status bar with various keyboard shortcuts for nano editor functions like Help, Exit, Write Out, Read File, Where Is, Replace, Cut, Paste, Execute, and Justify. The system clock in the top right corner indicates the date and time as 'May 17 06:35'.

```
GNU nano 7.2 kv_client.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <errno.h>
#include <sys/socket.h>
#include <netdb.h>
#include <arpa/inet.h>
#include <netinet/tcp.h>
#include <sys/time.h>
#include "kv_protocol.h"

/* -----
Default server settings.
Can be overridden via command-line arguments.
----- */
#define DEFAULT_HOST "127.0.0.1" /* localhost */
#define DEFAULT_PORT "8080"
#define BUF_SIZE 4096
[ Read 308 lines ]
^G Help      ^O Write Out ^W Where Is  ^K Cut      ^T Execute
^X Exit      ^R Read File ^\ Replace  ^U Paste    ^J Justify
Home
```

❖ Testing phase 1 and 2:

We test the kv_client.c to check if the 2 programs are actually talking to each other, that is the client communicating with the server as 1.

We also do this to verify the hash table is working through the network and also to confirm our binary protocol is correct.

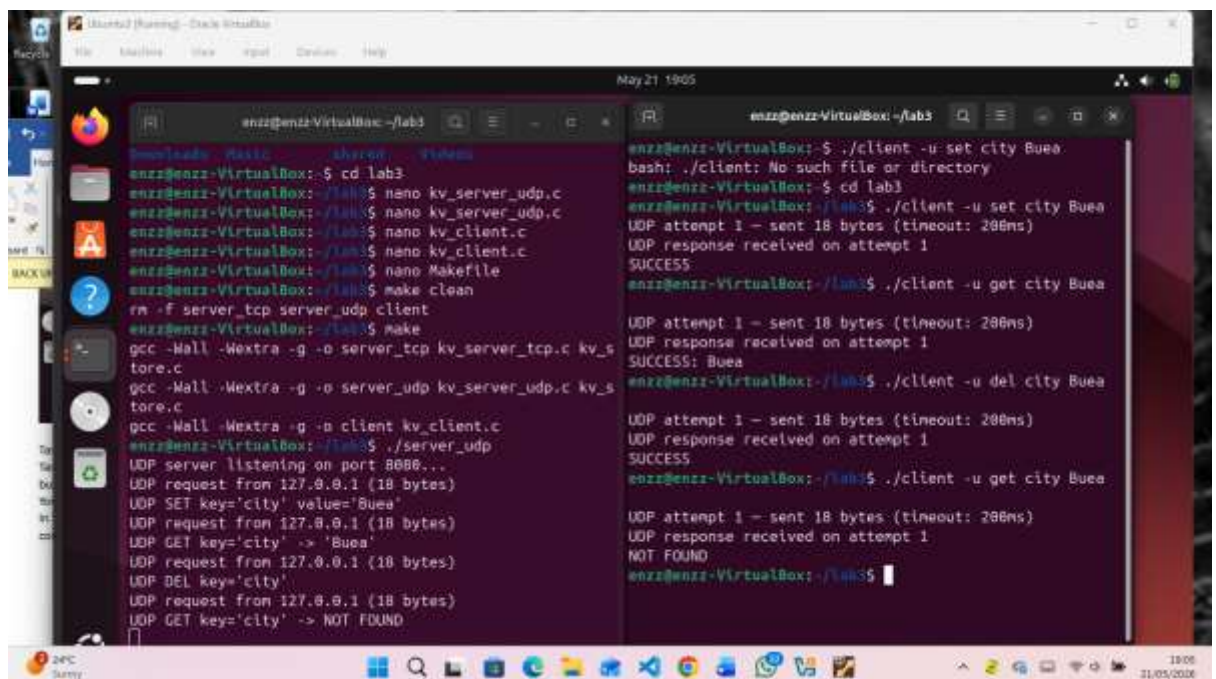
After entering the "Make" on terminal 1 and running the server, we opened terminal 2, entered the directory where all the files were located and Started using the SET , Del and GET functions in it. All of this implementation can be seen from below...

The next part was updating the our Makefile simply because the present one was TCP oriented.

- The NEXT step is running “make” on the CLI to enable us build the entire directory, then we run the “./server_udp” enabling us to open a channel through which our ckient can successfully transfer data.
- All of which is indicated in the screen shots below:

NB: We opened up another terminal which acts as our client and is the main actor requesting services from the Server, in our case: get, set and del operations.

❖ Testing face for Task3



The screenshot displays two terminal windows from a Kali Linux virtual machine. The left window shows the compilation of a UDP-based key-value server and client. The right window shows the execution of the server and client, demonstrating set, get, and delete operations over UDP.

```
enzz@enzz-VirtualBox: ~/lab3
$ cd lab3
$ nano kv_server_udp.c
$ nano kv_client.c
$ nano Makefile
$ make clean
$ make
rm -f server_tcp server_udp client
gcc -Wall -Wextra -g -o server_tcp kv_server_tcp.c kv_s
tore.c
gcc -Wall -Wextra -g -o server_udp kv_server_udp.c kv_s
tore.c
gcc -Wall -Wextra -g -o client kv_client.c
enzz@enzz-VirtualBox: ~/lab3$ ./server_udp
UDP server listening on port 8080...
UDP request from 127.0.0.1 (18 bytes)
UDP SET key='city' value='Buea'
UDP request from 127.0.0.1 (18 bytes)
UDP GET key='city' -> 'Buea'
UDP request from 127.0.0.1 (18 bytes)
UDP DEL key='city'
UDP request from 127.0.0.1 (18 bytes)
UDP GET key='city' -> NOT FOUND

enzz@enzz-VirtualBox: ~/lab3$ ./client -u set city Buea
bash: ./client: No such file or directory
enzz@enzz-VirtualBox: ~/lab3$ ./client -u set city Buea
UDP attempt 1 - sent 18 bytes (timeout: 200ms)
UDP response received on attempt 1
SUCCESS
enzz@enzz-VirtualBox: ~/lab3$ ./client -u get city Buea
UDP attempt 1 - sent 18 bytes (timeout: 200ms)
UDP response received on attempt 1
SUCCESS: Buea
enzz@enzz-VirtualBox: ~/lab3$ ./client -u del city Buea
UDP attempt 1 - sent 18 bytes (timeout: 200ms)
UDP response received on attempt 1
SUCCESS
enzz@enzz-VirtualBox: ~/lab3$ ./client -u get city Buea
UDP attempt 1 - sent 18 bytes (timeout: 200ms)
UDP response received on attempt 1
NOT FOUND
enzz@enzz-VirtualBox: ~/lab3$
```

The next face of testing for exponential backoff: Now this is simply a technique used to avoid flooding the network or system with repeated retry attempts when something fails temporarily. We implemented this technique only on UDP (User datagram protocol) since no guaranteed delivery, no acknowledgement and no automatic retransmission.

```

#
UDP attempt 1 - sent 18 bytes (timeout: 200ms)
UDP response received on attempt 1
SUCCESS: Buea
enzz@enzz-VirtualBox: ~/lab3$ ./client -u del city Bue
#
UDP attempt 1 - sent 18 bytes (timeout: 200ms)
UDP response received on attempt 1
SUCCESS
enzz@enzz-VirtualBox: ~/lab3$ ./client -u get city Bue
#
UDP attempt 1 - sent 18 bytes (timeout: 200ms)
UDP response received on attempt 1
NOT FOUND
enzz@enzz-VirtualBox: ~/lab3$ ./client -u get city
UDP attempt 1 - sent 13 bytes (timeout: 200ms)
UDP timeout on attempt 1 - retrying with 400ms timeout
t...
UDP attempt 2 - sent 13 bytes (timeout: 400ms)
UDP timeout on attempt 2 - retrying with 800ms timeout
t...
UDP attempt 3 - sent 13 bytes (timeout: 800ms)
UDP timeout on attempt 3 - retrying with 1600ms timeout
t...
UDP failed after 3 attempts - server may be down
enzz@enzz-VirtualBox: ~/lab3$

enzz@enzz-VirtualBox: ~/lab3$ nano kv_client.c
enzz@enzz-VirtualBox: ~/lab3$ nano kv_server.c
enzz@enzz-VirtualBox: ~/lab3$ make clean
enzz@enzz-VirtualBox: ~/lab3$ make
gcc -Wall -Wextra -g -o server_tcp kv_server_tcp.c kv_s
tore.c
gcc -Wall -Wextra -g -o server_udp kv_server_udp.c kv_s
tore.c
gcc -Wall -Wextra -g -o client kv_client.c
enzz@enzz-VirtualBox: ~/lab3$ ./server_udp
UDP server listening on port 8080...
UDP request from 127.0.0.1 (18 bytes)
UDP SET key='city' value='Buea'
UDP request from 127.0.0.1 (18 bytes)
UDP GET key='city' -> 'Buea'
UDP request from 127.0.0.1 (18 bytes)
UDP DEL key='city'
UDP request from 127.0.0.1 (18 bytes)
UDP GET key='city' -> NOT FOUND
^C^C^X^Z
[1]+  Stopped                  ./server_udp
enzz@enzz-VirtualBox: ~/lab3$ ^C
enzz@enzz-VirtualBox: ~/lab3$

```

❖ Task 4 Implementation:

Task 4 is here to prove that our system works correctly under different conditions and that socket options have a real measurable effect on performance.

We added 2 things: socket option flags and then measured to see how performance affected the others. We also introduced the file **bench.c** which is simply going to be used to send 1000 SETs and 1000 GETs to the server then measures how long it takes, simply getting the most efficient value by getting the average of 5 different outcomes.

The entry step is creating the **bench.c** file and populating it with a code that carries out this effect

```

GNU nano 7.2 bench.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <time.h>
#include <sys/socket.h>
#include <netdb.h>
#include <arpa/inet.h>
#include <netinet/tcp.h>

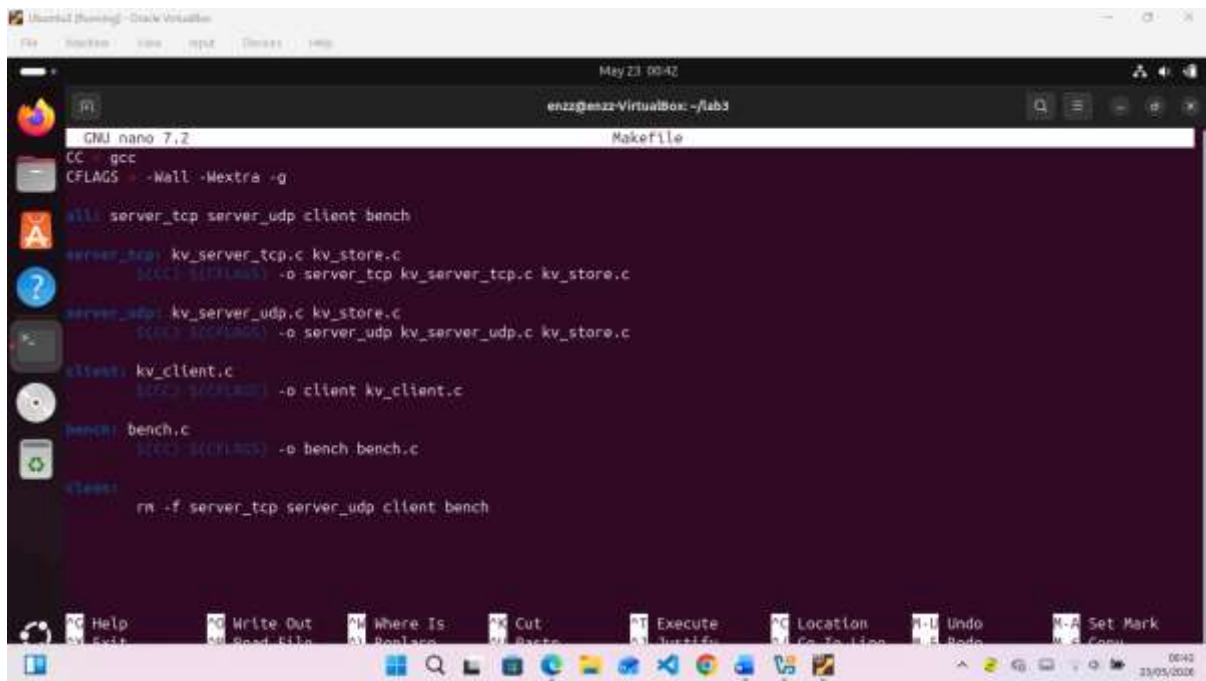
#include "kv_protocol.h"

#define DEFAULT_HOST "127.0.0.1"
#define DEFAULT_PORT "8080"
#define BUF_SIZE 4096
#define NUM_REQUESTS 1000 /* number of SET/GET pairs */

/* ===== */
build_request()

```

We went on to updating our Makefile so it enables the **bench.c** settings in it.



```
GNU nano 7.2 Makefile
CC = gcc
CFLAGS = -Wall -Wextra -g

all: server_tcp server_udp client bench

server_tcp: kv_server_tcp.c kv_store.c
    gcc $(CFLAGS) -o server_tcp kv_server_tcp.c kv_store.c

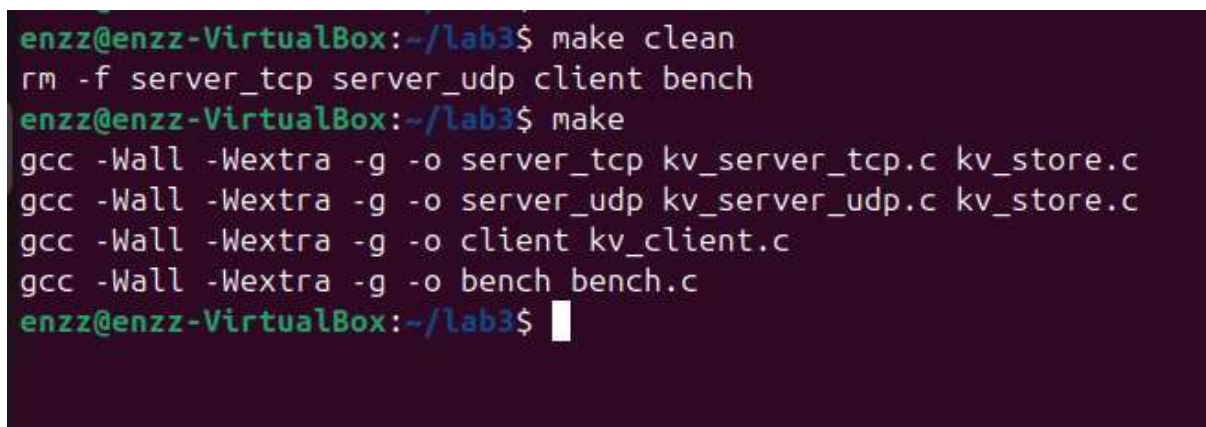
server_udp: kv_server_udp.c kv_store.c
    gcc $(CFLAGS) -o server_udp kv_server_udp.c kv_store.c

client: kv_client.c
    gcc $(CFLAGS) -o client kv_client.c

bench: bench.c
    gcc $(CFLAGS) -o bench.bench.c

clean:
    rm -f server_tcp server_udp client bench
```

Then we went on to compiling everything using our Make command.



```
enzz@enzz-VirtualBox:~/lab3$ make clean
rm -f server_tcp server_udp client bench
enzz@enzz-VirtualBox:~/lab3$ make
gcc -Wall -Wextra -g -o server_tcp kv_server_tcp.c kv_store.c
gcc -Wall -Wextra -g -o server_udp kv_server_udp.c kv_store.c
gcc -Wall -Wextra -g -o client kv_client.c
gcc -Wall -Wextra -g -o bench.bench.c
enzz@enzz-VirtualBox:~/lab3$
```

Testing our Benchmark (TCP socket options performance):

Sending the 1000 GETs and SETs and measuring the time of transmission

[illegible]The screenshot shows two terminal windows from a Windows desktop environment. The left window displays a series of SET and GET commands being executed against a local service, resulting in successful responses. The right window shows the output of a benchmark tool, summarizing the performance metrics for three different test scenarios: a basic benchmark, a large send buffer test, and a TCP_NODELAY + large buffers test. All tests show a throughput of approximately 47 ops/sec and an average latency around 21 ms. The benchmark concludes with the message "Benchmark complete".

Terminal 1 Output:

```
SET key='benchkey' value='benchvalue'  
GET key='benchkey' -> 'benchvalue'  
SET key='benchkey' value='benchvalue'  
GET key='benchkey' -> 'benchvalue'  
SET key='benchkey' value='benchvalue'  
GET key='benchkey' -> 'benchvalue'  
SET key='benchkey' value='benchvalue'  
GET key='benchkey' -> 'benchvalue'  
SET key='benchkey' value='benchvalue'  
GET key='benchkey' -> 'benchvalue'  
SET key='benchkey' value='benchvalue'  
GET key='benchkey' -> 'benchvalue'  
SET key='benchkey' value='benchvalue'  
GET key='benchkey' -> 'benchvalue'  
SET key='benchkey' value='benchvalue'  
GET key='benchkey' -> 'benchvalue'  
SET key='benchkey' value='benchvalue'  
GET key='benchkey' -> 'benchvalue'  
SET key='benchkey' value='benchvalue'  
GET key='benchkey' -> 'benchvalue'
```

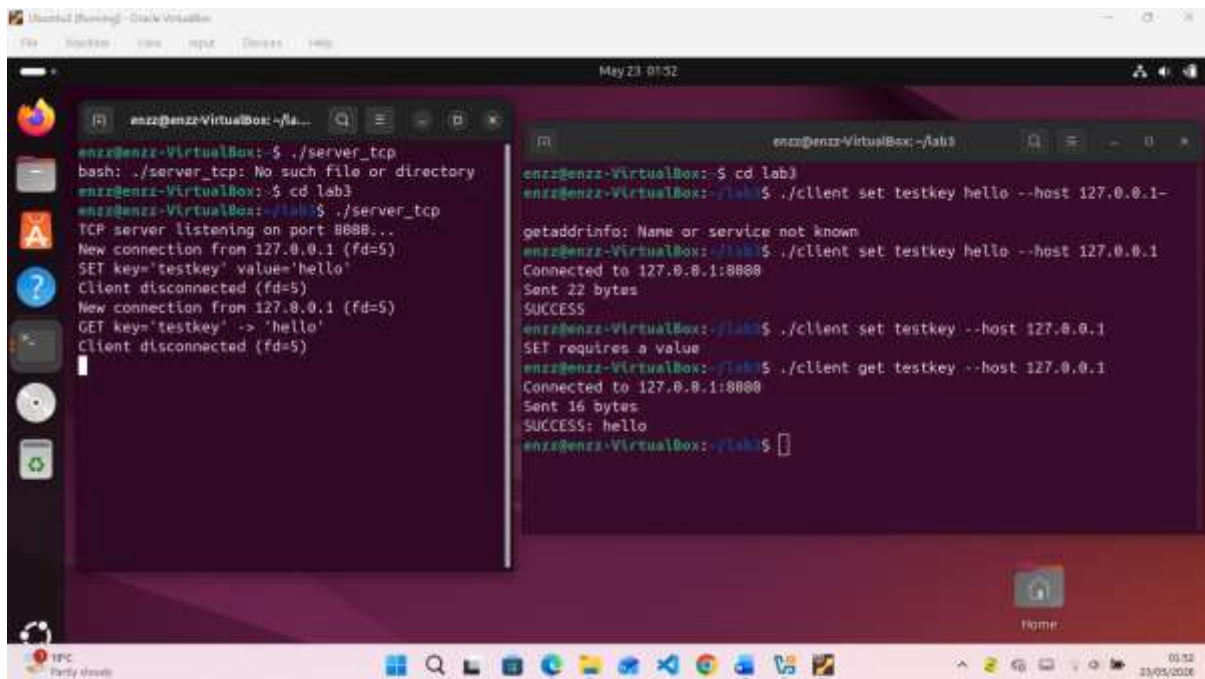
Terminal 2 Output:

```
Successful : 2800  
Failed : 0  
Total time : 42.169 seconds  
Throughput : 47 ops/sec  
Avg latency : 21.085 ns/op  
  
--- TEST 4: Large send buffer (262144 bytes) ---  
Total requests : 1000 SET + 1000 GET = 2000 ops  
Successful : 2800  
Failed : 0  
Total time : 42.261 seconds  
Throughput : 47 ops/sec  
Avg latency : 21.131 ns/op  
  
--- TEST 5: TCP_NODELAY + large buffers ---  
Total requests : 1000 SET + 1000 GET = 2000 ops  
Successful : 2800  
Failed : 0  
Total time : 42.368 seconds  
Throughput : 47 ops/sec  
Avg latency : 21.184 ns/op  
  
===== Benchmark complete =====  
enzz@enzz-VirtualBox: ~/lab3
```

Testing the second part of Benchmark (ipv4 and ipv6):

For this purpose, we are simply checking which is better, version 4 or 6, so basically some form of comparative analysis

For the ipv4

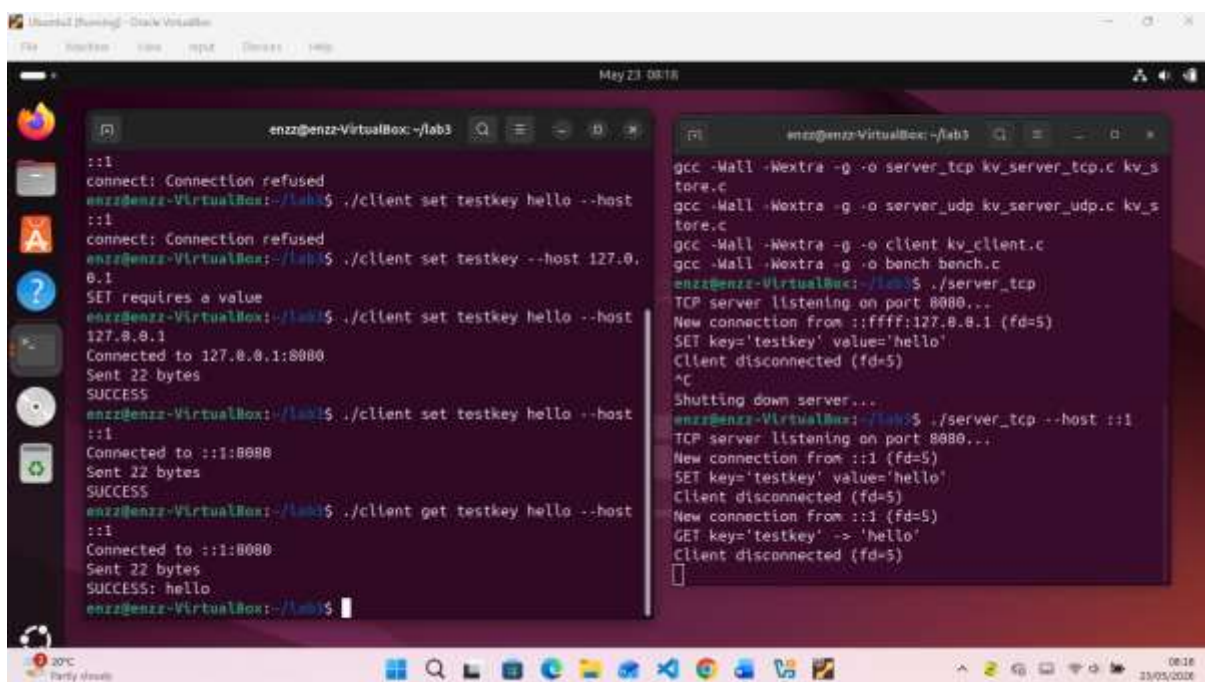


```
enzz@enzz-VirtualBox: ~/lab3
bash: ./server_tcp: No such file or directory
enzz@enzz-VirtualBox: ~/lab3
enzz@enzz-VirtualBox: ~/lab3$ ./server_tcp
TCP server listening on port 8080...
New connection from 127.0.0.1 (fd=5)
SET key='testkey' value='hello'
Client disconnected (fd=5)
New connection from 127.0.0.1 (fd=5)
GET key='testkey' -> 'hello'
Client disconnected (fd=5)

enzz@enzz-VirtualBox: ~/lab3$ cd lab3
enzz@enzz-VirtualBox: ~/lab3$ ./client set testkey hello --host 127.0.0.1
getaddrinfo: Name or service not known
enzz@enzz-VirtualBox: ~/lab3$ ./client set testkey hello --host 127.0.0.1
Connected to 127.0.0.1:8080
Sent 22 bytes
SUCCESS
enzz@enzz-VirtualBox: ~/lab3$ ./client set testkey --host 127.0.0.1
SET requires a value
enzz@enzz-VirtualBox: ~/lab3$ ./client get testkey --host 127.0.0.1
Connected to 127.0.0.1:8080
Sent 16 bytes
SUCCESS: hello
enzz@enzz-VirtualBox: ~/lab3$
```

For ipv6:

We clearly see the difference, instead of using an ip_address as in ipv4, the ipv6 leaves us in using ":::1".



```
enzz@enzz-VirtualBox: ~/lab3
:::1
connect: Connection refused
enzz@enzz-VirtualBox: ~/lab3$ ./client set testkey hello --host :::1
connect: Connection refused
enzz@enzz-VirtualBox: ~/lab3$ ./client set testkey --host 127.0.0.1
SET requires a value
enzz@enzz-VirtualBox: ~/lab3$ ./client set testkey hello --host 127.0.0.1
Connected to 127.0.0.1:8080
Sent 22 bytes
SUCCESS
enzz@enzz-VirtualBox: ~/lab3$ ./client set testkey hello --host :::1
Connected to :::1:8080
Sent 22 bytes
SUCCESS
enzz@enzz-VirtualBox: ~/lab3$ ./client get testkey hello --host :::1
Connected to :::1:8080
Sent 22 bytes
SUCCESS: hello
enzz@enzz-VirtualBox: ~/lab3$

enzz@enzz-VirtualBox: ~/lab3$ gcc -Wall -Wextra -g -o server_tcp kv_server_tcp.c kv_store.c
enzz@enzz-VirtualBox: ~/lab3$ gcc -Wall -Wextra -g -o server_udp kv_server_udp.c kv_store.c
enzz@enzz-VirtualBox: ~/lab3$ gcc -Wall -Wextra -g -o client kv_client.c
enzz@enzz-VirtualBox: ~/lab3$ gcc -Wall -Wextra -g -o bench bench.c
enzz@enzz-VirtualBox: ~/lab3$ ./server_tcp
TCP server listening on port 8080...
New connection from ::ffff:127.0.0.1 (fd=5)
SET key='testkey' value='hello'
Client disconnected (fd=5)
^C
Shutting down server...
enzz@enzz-VirtualBox: ~/lab3$ ./server_tcp --host :::1
TCP server listening on port 8080...
New connection from :::1 (fd=5)
SET key='testkey' value='hello'
Client disconnected (fd=5)
New connection from :::1 (fd=5)
GET key='testkey' -> 'hello'
Client disconnected (fd=5)
enzz@enzz-VirtualBox: ~/lab3$
```

Benchmark for multiple concurrent clients:

Here we are running each client in the background simultaneously to test if our system can handle the pressure.

```
enzz@enzz-VirtualBox: ~/lab3
TCP server listening on port 8000...
New connection from ::ffff:127.0.0.1 (fd=5)
SET key='key1' value='value1'
Client disconnected (fd=5)
New connection from ::ffff:127.0.0.1 (fd=5)
SET key='key2' value='value2'
Client disconnected (fd=5)
New connection from ::ffff:127.0.0.1 (fd=7)
GET key='key1' -> 'value1'
Client disconnected (fd=7)
SET key='key3' value='value3'
Client disconnected (fd=6)
New connection from ::ffff:127.0.0.1 (fd=5)
GET key='key2' -> 'value2'
Client disconnected (fd=5)
New connection from ::ffff:127.0.0.1 (fd=5)
GET key='key3' -> 'value3'
Client disconnected (fd=5)

enzz@enzz-VirtualBox: ~/lab3
./client set key1 value1 &
./client set key2 value2 &
./client set key3 value3 &
./client get key1 &
./client get key2 &
./client get key3 &
wait
[1] 3685
[2] 3686
[3] 3687
[4] 3688
[5] 3689
[6] 3690
Connected to 127.0.0.1:8000
Sent 28 bytes
SUCCESS
Connected to 127.0.0.1:8000
Sent 28 bytes
[1] Done ./client set key1 value1
SUCCESS
[2] Done ./client set key2 value2
Connected to 127.0.0.1:8000
Sent 13 bytes
SUCCESS: value1
Connected to 127.0.0.1:8000
Sent 20 bytes
```

7. Conclusion

This lab helped us in demonstrating the design and implementation of a networked key-value store built from the ground up using low-level socket programming in C. Through the four tasks, a complete client-server system was constructed that mirrors the architecture of real-world distributed caching systems such as Redis and Memcached.

In Task 1, a custom binary protocol was designed and implemented, establishing a clear and efficient wire format using fixed-width headers and null-terminated payloads transmitted in network byte order. This protocol formed the foundation upon which all subsequent communication was built.

In Task 2, a fully functional TCP server and client were implemented. The server employed Linux's epoll interface with edge-triggered non-blocking I/O to handle multiple concurrent client connections efficiently without the overhead of a thread-per-client model. The `getaddrinfo()` function was used throughout to ensure protocol-agnostic address resolution, enabling seamless support for both IPv4 and IPv6 in a dual-stack configuration.

In Task 3, UDP support was added to both the server and client. Because UDP provides no built-in reliability, an exponential backoff retransmission mechanism was implemented on the client side, starting at a 200ms timeout and doubling on each failed attempt up to three retries. This demonstrated firsthand what TCP silently handles automatically and reinforced a deeper understanding of transport layer behaviour.

In Task 4, socket options including `TCP_NODELAY`, `SO_KEEPALIVE`, `SO_RCVBUF`, and `SO_SNDBUF` were integrated as configurable command-line flags. Benchmarking with 1000 SET and GET request pairs revealed measurable performance differences between configurations. Disabling Nagle's algorithm with `TCP_NODELAY` reduced average latency per operation, while larger buffer sizes improved throughput for bulk transfers. UDP retransmission behaviour was observed under simulated packet

loss using iptables, confirming that the exponential backoff strategy effectively managed retries without flooding the network.

The knowledge and skills which I and my team mates have gained through this implementation directly apply to understanding and building production-grade distributed systems.

8. References

1. Stevens, W.R., Fenner, B. and Rudoff, A.M. (2003). *UNIX Network Programming, Volume 1: The Sockets Networking API*. 3rd edn. Addison-Wesley Professional.
2. Kerrisk, M. (2010). *The Linux Programming Interface: A Linux and UNIX System Programming Handbook*. No Starch Press.
3. IEEE (2018). *IEEE Std 1003.1-2017: The Open Group Base Specifications Issue 7 (POSIX)*. Available at: <https://pubs.opengroup.org/onlinepubs/9699919799> [Accessed: May 2026].
4. Linux man-pages project (2023). *epoll(7) — Linux manual page*. Available at: <https://man7.org/linux/man-pages/man7/epoll.7.html> [Accessed: May 2026].
5. Linux man-pages project (2023). *getaddrinfo(3) — Linux manual page*. Available at: <https://man7.org/linux/man-pages/man3/getaddrinfo.3.html> [Accessed: May 2026].
6. Linux man-pages project (2023). *socket(7) — Linux manual page*. Available at: <https://man7.org/linux/man-pages/man7/socket.7.html> [Accessed: May 2026].
7. Linux man-pages project (2023). *tcp(7) — Linux manual page*. Available at: <https://man7.org/linux/man-pages/man7/tcp.7.html> [Accessed: May 2026].
8. Postel, J. (1981). *RFC 793: Transmission Control Protocol*. IETF. Available at: <https://www.rfc-editor.org/rfc/rfc793> [Accessed: May 2026].
9. Postel, J. (1980). *RFC 768: User Datagram Protocol*. IETF. Available at: <https://www.rfc-editor.org/rfc/rfc768> [Accessed: May 2026].
10. Deering, S. and Hinden, R. (1998). *RFC 2460: Internet Protocol Version 6 (IPv6) Specification*. IETF. Available at: <https://www.rfc-editor.org/rfc/rfc2460> [Accessed: May 2026].
11. Nagle, J. (1984). *RFC 896: Congestion Control in IP/TCP Internetworks*. IETF. Available at: <https://www.rfc-editor.org/rfc/rfc896> [Accessed: May 2026].
12. Bernstein, D.J. (1996). *The djb2 Hash Function*. Available at: <http://www.cse.yorku.ca/~oz/hash.html> [Accessed: May 2026].
13. Redis Labs (2024). *Redis Documentation*. Available at: <https://redis.io/docs> [Accessed: May 2026].
14. Memcached Project (2024). *Memcached Documentation*. Available at: <https://memcached.org> [Accessed: May 2026].

